키매니저 워커 (Key Manager Worker)

소개

키매니저 워커는 Oasis Core의 런타임 계층에서 암호화 키 관리를 담당하는 특수한 워커입니다. 키매니저 워커는 다른 런타임들이 사용할 수 있는 마스터 시크릿과 임시 시크릿을 안전하게 생성, 저장, 배포하는 역할을 수행합니다.

키매니저 워커는 TEE(Trusted Execution Environment) 내에서 실행되어 높은 수준의 보안을 제공하며, 분산 키 생성 프로토콜을 통해 단일 실패 지점을 방지합니다.

주요 특징

- **분산 키 생성**: 여러 키매니저 노드가 협력하여 키 생성
- **TEE 보안**: SGX 엔클레이브 내에서 키 처리
- **키 회전**: 정기적인 키 업데이트 지원
- **접근 제어**: 정책 기반 키 접근 관리
- **CHURP 지원**: 분산 키 재분배 프로토콜

아키텍처

키매니저 워커는 자체 런타임을 가지며 다른 런타임들의 실행자 위원회를 추적하여 접근 제어 목록을 업데이트합니다.

주요 컴포넌트

1. **키매니저 워커**: 전체 키 관리 프로세스 조정

2. **시크릿 워커**: 마스터/임시 시크릿 생성 및 관리

3. **CHURP 워커**: 분산 키 재분배 프로토콜 실행

4. **접근 제어**: 클라이언트 인증 및 권한 관리

5. **키매니저 엔클레이브**: TEE 내 키 처리 로직##

키매니저 워커 구현

워커 구조체

```
// Worker는 키매니저 워커입니다

// 자체 런타임을 가지며 다른 런타임들의 실행자 위원회를 추적하여

// 접근 제어 목록을 업데이트합니다

type Worker struct {

sync.RWMutex

*runtimeRegistry.RuntimeHostNode

logger *logging.Logger

ctx context.Context

cancelCtx context.CancelFunc

quitCh  chan  struct{}

initCh  chan  struct{}

nodeID signature.PublicKey

runtime runtimeRegistry.Runtime

runtimeID common.Namespace

runtimeLabel  string

// 하위 워커들
```

```
kmNodeWatcher *kmNodeWatcher

kmRuntimeWatcher *kmRuntimeWatcher

secretsWorker *secretsWorker

churpWorker *churpWorker

// 접근 제어

accessControllers []workerKeymanager.RPCAccessController

accessControllersByMethod  map[string]workerKeymanager.RPCAccessController

peerMap *PeerMap

accessList *AccessList

// 공통 서비스

commonWorker *workerCommon.Worker

roleProvider registration.RoleProvider

keymanager api.Backend

keyManagerClient *commonCommittee.KeyManagerClientWrapper

services *service.Group

enabled  bool

}
```

워커 생명주기

```
// Start는 키매니저 워커를 시작합니다

func (w *Worker) Start() error {

if !w.enabled {
```

```
w.logger.Info("키매니저 워커가 비활성화되어 시작하지 않음")
close(w.initCh)
return nil
}
go w.worker()
return nil
}


// worker는 메인 워커 루프입니다
func (w *Worker) worker() {
w.logger.Info("키매니저 워커 시작")
defer close(w.quitCh)
// 합의 동기화 대기
w.logger.Info("합의가 초기 동기화를 완료할 때까지 대기")
select {
case <-w.ctx.Done():
return
case <-w.commonWorker.Consensus.Synced():
}
w.logger.Info("합의가 초기 동기화를 완료함")
// 런타임 컴포넌트 발견 대기
var comp *bundle.ExplodedComponent
if ok := w.discoverRuntimeComponent(&comp); !ok {
```

```go
return
}
// 런타임 컴포넌트 프로비저닝
if err := w.provisionRuntime(comp); err != nil {
w.logger.Error("런타임 컴포넌트 프로비저닝 실패", "err", err)
return
}
// 하위 워커들과 서비스 시작
w.startSubWorkers()
close(w.initCh)
}


// discoverRuntimeComponent는 런타임 컴포넌트를 발견합니다
func (w *Worker) discoverRuntimeComponent(comp **bundle.ExplodedComponent) bool
bundleRegistry := w.commonWorker.RuntimeRegistry.GetBundleRegistry()
compCh, compSub := bundleRegistry.WatchComponents(w.runtimeID)
defer compSub.Close()
comps := bundleRegistry.Components(w.runtimeID)
switch numComps := len(comps); numComps {
case 0:
w.logger.Info("런타임 컴포넌트 발견 대기")
for {
select {
```

```
case  compNotify := <-compCh:
if  compNotify.Added == nil {
continue
}
*comp = compNotify.Added
return true
case <-w.ctx.Done():
return false
}
}
case  1:
*comp = comps[0]
default:
w.logger.Error("하나의 런타임 컴포넌트가 예상됨", "total", numComps)
return false
}
if !(*comp).ID().IsRONL() {
w.logger.Error("RONL 런타임 컴포넌트가 예상됨", "id", (*comp).ID())
return false
}
w.logger.Info("런타임 컴포넌트 발견됨",
"id", (*comp).ID(),
"version", (*comp).Version,
```

```
)
return true
}


// provisionRuntime은 런타임을 프로비저닝합니다
func (w *Worker) provisionRuntime(comp *bundle.ExplodedComponent) error {
w.logger.Info("런타임 컴포넌트 프로비저닝",
"id", comp.ID(),
"version", comp.Version,
)
if err := w.ProvisionHostedRuntimeComponent(comp); err != nil {
return fmt.Errorf("런타임 컴포넌트 프로비저닝 실패: %w", err)
}
// 런타임을 지정된 버전으로 설정
w.SetHostedRuntimeVersion(&comp.Version, nil)
// 런타임 시작
hrt := w.GetHostedRuntime()
hrt.Start()
// 런타임 버전이 활성화되었는지 확인
if _, err := w.GetHostedRuntimeActiveVersion(); err != nil {
return fmt.Errorf("런타임 컴포넌트 활성화 실패: %w", err)
}
return nil
```

```go
}

// startSubWorkers는 하위 워커들을 시작합니다
func (w *Worker) startSubWorkers() {
var wg sync.WaitGroup
defer wg.Wait()
wg.Add(5)
// 서비스 그룹 시작
go func() {
defer wg.Done()
if err := w.services.Serve(w.ctx); err != nil {
w.logger.Error("서비스 그룹 중지됨", "err", err)
}
}()
// 키매니저 노드 감시자
go func() {
defer wg.Done()
w.kmNodeWatcher.watch(w.ctx)
}()
// 키매니저 런타임 감시자
go func() {
defer wg.Done()
w.kmRuntimeWatcher.watch(w.ctx)
```

```
}()
// 시크릿 워커
go func() {
defer wg.Done()
hrt := w.GetHostedRuntime()
w.secretsWorker.work(w.ctx, hrt)
}()
// CHURP 워커
go func() {
defer wg.Done()
hrt := w.GetHostedRuntime()
w.churpWorker.work(w.ctx, hrt)
}()
}
```

시크릿 관리

시크릿 워커 구조

```
type secretsWorker struct {
mu sync.RWMutex
logger *logging.Logger
initCh chan struct{}
```

```
runtimeID common.Namespace

runtimeLabel  string

privatePeers  map[core.PeerID]struct{}

kmWorker *Worker

commonWorker *workerCommon.Worker

roleProvider registration.RoleProvider

keymanager api.Backend

status workerKm.SecretsStatus

kmStatus *secrets.Status

// 엔클레이브 초기화

initEnclaveInProgress  bool

initEnclaveRequired  bool

initEnclaveDoneCh  chan *secrets.SignedInitResponse

initEnclaveRetryCh <-chan time.Time

initEnclaveRetryTicker *backoff.Ticker

// 마스터 시크릿

mstSecret *secrets.SignedEncryptedMasterSecret

loadMstSecRetry  int

genMstSecDoneCh  chan  bool

genMstSecEpoch beacon.EpochTime

genMstSecInProgress  bool

genMstSecRetry  int

// 임시 시크릿
```

```
ephSecret *secrets.SignedEncryptedEphemeralSecret
loadEphSecRetry int
genEphSecDoneCh chan bool
genEphSecInProgress bool
genEphSecRetry int
genSecHeight int64
}
```

마스터 시크릿 생성

```
// generateMasterSecret은 마스터 시크릿을 생성합니다
func (w *secretsWorker) generateMasterSecret(
ctx context.Context,
runtimeID common.Namespace,
height int64,
generation uint64,
epoch beacon.EpochTime,
kmStatus *secrets.Status,
) error {
w.logger.Info("마스터 시크릿 생성",
"height", height,
"generation", generation,
"epoch", epoch,
```

```
)
// 이 에포크에서 마스터 시크릿이 제안되었는지 확인
lastSecret, err := w.commonWorker.Consensus.KeyManager().Secrets().GetMasterSecr
Height: consensus.HeightLatest,
ID: runtimeID,
})
if err != nil && err != secrets.ErrNoSuchMasterSecret {
return fmt.Errorf("마스터 시크릿 조회 실패: %w", err)
}
if lastSecret != nil && epoch == lastSecret.Secret.Epoch {
return fmt.Errorf("마스터 시크릿은 에포크당 한 번만 제안 가능")
}
// 회전이 허용되는지 확인
if err = kmStatus.VerifyRotationEpoch(epoch); err != nil {
return fmt.Errorf("회전 에포크 검증 실패: %w", err)
}
// 노드가 키매니저 위원회에 있는지 확인
id := w.commonWorker.Identity.NodeSigner.Public()
if !slices.Contains(kmStatus.Nodes, id) {
w.logger.Info("마스터 시크릿 생성 건너뜀, 노드가 키매니저 위원회에 없음")
return fmt.Errorf("노드가 키매니저 위원회에 없음")
}
// 마스터 시크릿 생성
```

```
args := secrets.GenerateMasterSecretRequest{
Generation: generation,
Epoch: epoch,
}
var  rsp secrets.GenerateMasterSecretResponse
if  err = w.kmWorker.callEnclaveLocal(ctx, secrets.RPCMethodGenerateMasterSecret
return  fmt.Errorf("마스터 시크릿 생성 실패: %w", err)
}
// 응답 검증
rak, err := w.kmWorker.runtimeAttestationKey()
if  err != nil {
return  fmt.Errorf("런타임 증명 키 획득 실패: %w", err)
}
reks, err := w.kmWorker.runtimeEncryptionKeys(kmStatus.Nodes)
if  err != nil {
return  fmt.Errorf("런타임 암호화 키 획득 실패: %w", err)
}
if  err = rsp.SignedSecret.Verify(generation, epoch, reks, rak);  err != nil {
return  fmt.Errorf("마스터 시크릿 서명 검증 실패: %w", err)
}
// 트랜잭션 게시
tx := secrets.NewPublishMasterSecretTx(0, nil, &rsp.SignedSecret)
if  err = consensus.SignAndSubmitTx(ctx, w.commonWorker.Consensus, w.commonWorke
```

```go
return  fmt.Errorf("마스터 시크릿 트랜잭션 제출 실패: %w", err)
}
w.logger.Info("마스터 시크릿 생성 완료",
"generation", rsp.SignedSecret.Secret.Generation,
"epoch", rsp.SignedSecret.Secret.Epoch,
)
return  nil
}


// loadMasterSecret은 마스터 시크릿을 로드합니다
func (w *secretsWorker) loadMasterSecret(ctx context.Context, sigSecret *secrets
w.logger.Info("마스터 시크릿 로딩",
"generation", sigSecret.Secret.Generation,
"epoch", sigSecret.Secret.Epoch,
)
args := secrets.LoadMasterSecretRequest{
SignedSecret: *sigSecret,
}
var  rsp protocol.Empty
if  err := w.kmWorker.callEnclaveLocal(ctx, secrets.RPCMethodLoadMasterSecret, a
return  fmt.Errorf("마스터 시크릿 로드 실패: %w", err)
}
w.logger.Info("마스터 시크릿 로드 완료")
```

```
return  nil
}
```

임시 시크릿 관리

```
// generateEphemeralSecret은 임시 시크릿을 생성합니다
func (w *secretsWorker) generateEphemeralSecret(
ctx context.Context,
runtimeID common.Namespace,
height  int64,
epoch beacon.EpochTime,
kmStatus *secrets.Status,
) error {
w.logger.Info("임시 시크릿 생성",
"height", height,
"epoch", epoch,
)
// 이 에포크에서 임시 시크릿이 제안되었는지 확인
lastSecret, err := w.commonWorker.Consensus.KeyManager().Secrets().GetEphemeralS
Height: consensus.HeightLatest,
ID: runtimeID,
})
```

```go
if err != nil && err != secrets.ErrNoSuchEphemeralSecret {
return fmt.Errorf("임시 시크릿 조회 실패: %w", err)
}
if lastSecret != nil && epoch == lastSecret.Secret.Epoch {
return fmt.Errorf("임시 시크릿은 에포크당 한 번만 제안 가능")
}
// 노드가 키매니저 위원회에 있는지 확인
id := w.commonWorker.Identity.NodeSigner.Public()
if !slices.Contains(kmStatus.Nodes, id) {
w.logger.Info("임시 시크릿 생성 건너뜀, 노드가 키매니저 위원회에 없음")
return fmt.Errorf("노드가 키매니저 위원회에 없음")
}
// 임시 시크릿 생성
args := secrets.GenerateEphemeralSecretRequest{
Epoch: epoch,
}
var rsp secrets.GenerateEphemeralSecretResponse
if err = w.kmWorker.callEnclaveLocal(ctx, secrets.RPCMethodGenerateEphemeralSec
return fmt.Errorf("임시 시크릿 생성 실패: %w", err)
}
// 응답 검증
rak, err := w.kmWorker.runtimeAttestationKey()
if err != nil {
```

```
return  fmt.Errorf("런타임 증명 키 획득 실패: %w", err)
}
reks, err := w.kmWorker.runtimeEncryptionKeys(kmStatus.Nodes)
if  err != nil {
return  fmt.Errorf("런타임 암호화 키 획득 실패: %w", err)
}
if  err = rsp.SignedSecret.Verify(epoch, reks, rak);  err != nil {
return  fmt.Errorf("임시 시크릿 서명 검증 실패: %w", err)
}
// 트랜잭션 게시
tx := secrets.NewPublishEphemeralSecretTx(0, nil, &rsp.SignedSecret)
if  err = consensus.SignAndSubmitTx(ctx, w.commonWorker.Consensus, w.commonWorke
return  fmt.Errorf("임시 시크릿 트랜잭션 제출 실패: %w", err)
}
w.logger.Info("임시 시크릿 생성 완료", "epoch", rsp.SignedSecret.Secret.Epoch)
return  nil
}


// loadEphemeralSecret은 임시 시크릿을 로드합니다
func (w *secretsWorker) loadEphemeralSecret(ctx context.Context, sigSecret *secr
w.logger.Info("임시 시크릿 로딩", "epoch", sigSecret.Secret.Epoch)
args := secrets.LoadEphemeralSecretRequest{
SignedSecret: *sigSecret,
```

```
}
var  rsp protocol.Empty
if  err := w.kmWorker.callEnclaveLocal(ctx, secrets.RPCMethodLoadEphemeralSecret
return  fmt.Errorf("임시 시크릿 로드 실패: %w", err)
}
w.logger.Info("임시 시크릿 로드 완료")
return  nil
}
```

접근 제어

RPC 접근 제어

```
// RPCAccessController는 RPC 접근을 제어하는 인터페이스입니다
type RPCAccessController interface {
// Methods는 이 컨트롤러가 처리하는 메서드 목록을 반환합니다
Methods() []string
// Connect는 피어가 연결할 수 있는지 확인합니다
Connect(ctx context.Context, peerID core.PeerID) bool
// Authorize는 특정 메서드 호출을 승인합니다
Authorize(ctx context.Context, method  string, kind enclaverpc.Kind, peerID core
}
```

```go
// Connect는 피어 연결을 승인합니다
func (w *secretsWorker) Connect(ctx context.Context, peerID core.PeerID) bool {
// 초기화 후 요청 수락 시작
w.mu.RLock()
state := w.status.Worker.Status
kmStatus := w.status.Status
w.mu.RUnlock()
if state != workerKm.StatusStateReady || kmStatus == nil {
return false
}
// 프라이빗 피어는 제한 없이 보안 메서드에 접근 가능
if _, ok := w.privatePeers[peerID]; ok {
return true
}
// 다른 피어들은 승인 과정을 거쳐야 함
if err := w.authorizeNode(ctx, peerID, kmStatus); err == nil {
return true
}
if err := w.authorizeKeyManager(peerID); err == nil {
return true
}
return false
}
```

```go
// Authorize는 메서드 호출을 승인합니다
func (w *secretsWorker) Authorize(ctx context.Context, method  string, kind encl
// 초기화 후 요청 수락 시작
w.mu.RLock()
state := w.status.Worker.Status
kmStatus := w.status.Status
w.mu.RUnlock()
if  state != workerKm.StatusStateReady || kmStatus == nil {
return  fmt.Errorf("초기화되지 않음")
}
// 메서드가 지원되는지 확인
switch  kind {
case  enclaverpc.KindInsecureQuery:
if  _, ok := insecureRPCMethods[method]; !ok {
return  fmt.Errorf("지원하지 않는 메서드: %s", method)
}
return  nil
case  enclaverpc.KindNoiseSession:
if  _, ok := secureRPCMethods[method]; !ok {
return  fmt.Errorf("지원하지 않는 메서드: %s", method)
}
default:
```

```go
return fmt.Errorf("지원하지 않는 종류: %s", kind)
}
// 프라이빗 피어는 제한 없이 보안 메서드에 접근 가능
if _, ok := w.privatePeers[peerID]; ok {
return nil
}
// 다른 피어들은 승인 과정을 거쳐야 함
switch method {
case secrets.RPCMethodGetOrCreateKeys, secrets.RPCMethodGetOrCreateEphemeralKey
return w.authorizeNode(ctx, peerID, kmStatus)
case secrets.RPCMethodReplicateMasterSecret, secrets.RPCMethodReplicateEphemera
return w.authorizeKeyManager(peerID)
default:
return fmt.Errorf("지원하지 않는 메서드: %s", method)
}
}


// authorizeNode는 노드를 승인합니다
func (w *secretsWorker) authorizeNode(ctx context.Context, peerID core.PeerID, k
rt, err := w.kmWorker.runtime.RegistryDescriptor(ctx)
if err != nil {
return fmt.Errorf("런타임 디스크립터 획득 실패: %w", err)
}
```

```
switch rt.TEEHardware {
case node.TEEHardwareInvalid:
// 비보안 키매니저 엔클레이브는 모든 런타임이 쿼리 가능 (테스트용)
if kmStatus.IsSecure {
return fmt.Errorf("신뢰할 수 없는 하드웨어")
}
return nil
case node.TEEHardwareIntelSGX:
// 보안 키매니저 엔클레이브는 정책에 지정된 런타임만 쿼리 가능
if kmStatus.Policy == nil {
return fmt.Errorf("정책이 설정되지 않음")
}
rts := w.kmWorker.accessList.Runtimes(peerID)
for _, enc := range kmStatus.Policy.Policy.Enclaves {
for rtID := range enc.MayQuery {
if rts.Contains(rtID) {
return nil
}
}
}
return fmt.Errorf("쿼리가 허용되지 않음")
default:
return fmt.Errorf("지원하지 않는 하드웨어: %s", rt.TEEHardware)
```

```
}
}

// authorizeKeyManager는 키매니저를 승인합니다
func (w *secretsWorker) authorizeKeyManager(peerID core.PeerID) error {
// 같은 키매니저 런타임 내의 피어만 허용
if !w.kmWorker.accessList.Runtimes(peerID).Contains(w.runtimeID) {
return  fmt.Errorf("키매니저가 아님")
}
return  nil
}
```

CHURP 프로토콜

CHURP 워커

```
// churpWorker는 설정된 스킴에 대해 CHURP 프로토콜을 실행합니다
type churpWorker struct {
logger *logging.Logger
runtimeID common.Namespace
runtimeLabel  string
kmWorker *Worker
commonWorker *workerCommon.Worker
```

```go
roleProvider registration.RoleProvider
keymanager api.Backend
initCh chan struct{}
status workerKm.ChurpStatus
}

// work는 CHURP 워커의 메인 루프입니다
func (w *churpWorker) work(ctx context.Context, hrt host.Runtime) {
w.logger.Info("CHURP 워커 시작")
// CHURP 상태 업데이트 구독
statusCh, statusSub, err := w.keymanager.Churp().WatchStatuses(ctx)
if err != nil {
w.logger.Error("CHURP 상태 감시 실패", "err", err)
return
}
defer statusSub.Close()
// 에포크 전환 구독
epoCh, epoSub, err := w.commonWorker.Consensus.Beacon().WatchLatestEpoch(ctx)
if err != nil {
w.logger.Error("에포크 감시 실패", "err", err)
return
}
defer epoSub.Close()
```

```go
// 블록 높이 감시
blkCh, blkSub, err := w.commonWorker.Consensus.Core().WatchBlocks(ctx)
if err != nil {
w.logger.Error("블록 감시 실패", "err", err)
return
}
defer blkSub.Close()
close(w.initCh)
for {
select {
case epoch := <-epoCh:
w.handleNewEpoch(ctx, epoch)
case blk := <-blkCh:
w.handleNewBlock(ctx, blk)
case status := <-statusCh:
w.handleStatusUpdate(ctx, status)
case <-ctx.Done():
return
}
}
}

// handleNewEpoch는 새 에포크를 처리합니다
```

```go
func (w *churpWorker) handleNewEpoch(ctx context.Context, epoch beacon.EpochTime

w.logger.Debug("새 에포크 처리", "epoch", epoch)

// CHURP 스킴 상태 확인 및 필요시 키 재분배 시작

statuses := w.keymanager.Churp().Statuses()

for _, status := range statuses {

if !status.ID.Equal(&w.runtimeID) {

continue

}

if w.shouldStartHandoff(status, epoch) {

w.startHandoff(ctx, status, epoch)

}

}

}



// shouldStartHandoff는 핸드오프를 시작해야 하는지 확인합니다

func (w *churpWorker) shouldStartHandoff(status *churp.Status, epoch beacon.Epoc

// 핸드오프 간격 확인

if status.HandoffInterval == 0 {

return false // 핸드오프 비활성화

}

nextHandoff := status.NextHandoff

return epoch >= nextHandoff

}
```

```go
// startHandoff는 키 핸드오프를 시작합니다
func (w *churpWorker) startHandoff(ctx context.Context, status *churp.Status, ep
w.logger.Info("CHURP 핸드오프 시작",
"scheme_id", status.ID,
"epoch", epoch,
)
// 엔클레이브에서 핸드오프 실행
args := churp.HandoffRequest{
ID: status.ID,
Epoch: epoch,
}
var  rsp churp.HandoffResponse
if  err := w.kmWorker.callEnclaveLocal(ctx, churp.RPCMethodHandoff, args, &rsp);
w.logger.Error("CHURP 핸드오프 실패",
"err", err,
"scheme_id", status.ID,
"epoch", epoch,
)
return
}
w.logger.Info("CHURP 핸드오프 완료",
"scheme_id", status.ID,
```

```
"epoch", epoch,
)
}
```

엔클레이브 통신

엔클레이브 호출

```
// CallEnclave는 엔클레이브를 호출합니다
func (w *Worker) CallEnclave(ctx context.Context, data []byte, kind enclaverpc.K
// 프레임/요청 데이터를 살펴보고 메서드 추출
var method string
switch kind {
case enclaverpc.KindNoiseSession:
var frame enclaverpc.Frame
if err := cbor.Unmarshal(data, &frame); err != nil {
return nil, fmt.Errorf("잘못된 RPC 프레임")
}
method = frame.UntrustedPlaintext
case enclaverpc.KindInsecureQuery:
var req enclaverpc.Request
if err := cbor.Unmarshal(data, &req); err != nil {
return nil, fmt.Errorf("잘못된 RPC 요청")
```

```
}
method = req.Method
default:
return  nil, fmt.Errorf("지원하지 않는 RPC 종류")
}
// 접근 제어 처리
peerID, ok := rpc.PeerIDFromContext(ctx)
if !ok {
return  nil, fmt.Errorf("승인되지 않음: 알 수 없는 피어")
}
switch {
case  method == api.RPCMethodConnect && kind == enclaverpc.KindNoiseSession:
// 적어도 하나의 컨트롤러가 승인하면 연결 허용
fn := func(ctrl workerKeymanager.RPCAccessController) bool {
return  ctrl.Connect(ctx, peerID)
}
if !slices.ContainsFunc(w.accessControllers, fn) {
return  nil, fmt.Errorf("연결 승인되지 않음")
}
default:
ctrl, ok := w.accessControllersByMethod[method]
if !ok {
return  nil, fmt.Errorf("지원하지 않는 RPC 메서드")
```

```
}
if err := ctrl.Authorize(ctx, method, kind, peerID); err != nil {
return nil, fmt.Errorf("승인되지 않음: %w", err)
}
}
ctx, cancel := context.WithTimeout(ctx, rpcCallTimeout)
defer cancel()
req := &protocol.Body{
RuntimeRPCCallRequest: &protocol.RuntimeRPCCallRequest{
Request: data,
Kind: kind,
PeerID: []byte(peerID),
},
}
rt := w.GetHostedRuntime()
if rt == nil {
return nil, fmt.Errorf("초기화되지 않음")
}
response, err := rt.Call(ctx, req)
if err != nil {
w.logger.Error("런타임에 RPC 호출 전달 실패",
"err", err,
"kind", kind,
```

```
)
return nil, fmt.Errorf("런타임 호출 실패: %w", err)
}
resp := response.RuntimeRPCCallResponse
if resp == nil {
w.logger.Error("런타임에서 잘못된 응답", "response", response)
return nil, fmt.Errorf("런타임에서 잘못된 응답")
}
return resp.Response, nil
}


// callEnclaveLocal은 로컬 엔클레이브 호출을 수행합니다
func (w *Worker) callEnclaveLocal(ctx context.Context, method string, args any,
rt := w.GetHostedRuntime()
if rt == nil {
return fmt.Errorf("초기화되지 않음")
}
return host.NewRichRuntime(rt).LocalRPC(ctx, method, args, rsp)
}
```

보안 고려사항

TEE 보안

```go
// runtimeAttestationKey는 런타임 증명 키를 반환합니다
func (w *Worker) runtimeAttestationKey() (*signature.PublicKey, error) {
kmRt, err := w.runtime.RegistryDescriptor(w.ctx)
if err != nil {
return nil, fmt.Errorf("런타임 디스크립터 획득 실패: %w", err)
}
var rak *signature.PublicKey
switch kmRt.TEEHardware {
case node.TEEHardwareInvalid:
// 비보안 모드 (테스트용)
rak = &api.InsecureRAK
case node.TEEHardwareIntelSGX:
// SGX 모드
capabilityTEE, err := w.GetHostedRuntimeCapabilityTEE()
if err != nil {
return nil, fmt.Errorf("TEE 능력 가져오기 실패: %w", err)
}
if capabilityTEE == nil {
return nil, fmt.Errorf("런타임이 TEE 내에서 실행되지 않음")
}
rak = &capabilityTEE.RAK
default:
```

```
return nil, fmt.Errorf("TEE 하드웨어 불일치")
}
return rak, nil
}


// runtimeEncryptionKeys는 런타임 암호화 키들을 반환합니다
func (w *Worker) runtimeEncryptionKeys(nodes []signature.PublicKey) (map[x25519.
kmRt, err := w.runtime.RegistryDescriptor(w.ctx)
if err != nil {
return nil, fmt.Errorf("런타임 디스크립터 획득 실패: %w", err)
}
reks := make(map[x25519.PublicKey]struct{})
for _, id := range nodes {
var n *node.Node
n, err := w.commonWorker.Consensus.Registry().GetNode(w.ctx, &registry.IDQuery{
Height: consensus.HeightLatest,
ID: id,
})
switch err {
case nil:
case registry.ErrNoSuchNode:
continue
default:
```

```go
return nil, fmt.Errorf("노드 정보 획득 실패: %w", err)
}
// 키매니저 런타임을 실행하는 노드 찾기
idx := slices.IndexFunc(n.Runtimes, func(rt *node.Runtime) bool {
return rt.ID.Equal(&w.runtimeID)
})
if idx == -1 {
continue
}
nRt := n.Runtimes[idx]
var rek x25519.PublicKey
switch kmRt.TEEHardware {
case node.TEEHardwareInvalid:
rek = api.InsecureREK
case node.TEEHardwareIntelSGX:
if nRt.Capabilities.TEE == nil || nRt.Capabilities.TEE.REK == nil {
continue
}
rek = *nRt.Capabilities.TEE.REK
default:
continue
}
reks[rek] = struct{}{}
```

```
}
return reks, nil
}
```

키 회전 보안

```
// selectBlockHeight는 지정된 에포크의 백분위수 내에서 랜덤 블록을 선택합니다
func (w *Worker) selectBlockHeight(epoch beacon.EpochTime, from uint8, to uint
// 현재 에포크의 첫 번째 블록 높이 가져오기
now, err := w.commonWorker.Consensus.Beacon().GetEpoch(w.ctx, consensus.HeightLa
if err != nil {
return 0, fmt.Errorf("에포크 가져오기 실패: %w", err)
}
first, err := w.commonWorker.Consensus.Beacon().GetEpochBlock(w.ctx, now)
if err != nil {
return 0, fmt.Errorf("에포크 블록 높이 가져오기 실패: %w", err)
}
// 현재 에포크의 에포크 간격 가져오기
params, err := w.commonWorker.Consensus.Beacon().ConsensusParameters(w.ctx, cons
if err != nil {
return 0, fmt.Errorf("합의 매개변수 가져오기 실패: %w", err)
}
interval := params.Interval()
```

```
// 에포크 시간을 모킹할 때는 간격을 0으로 사용
if  cmdFlags.DebugDontBlameOasis() && params.DebugMockBackend {
interval = 0
}
// 주어진 백분위수에서 랜덤 블록 선택
offset := interval * int64(from) / 100
span := interval * int64(to-from) / 100
span = max(1, span)
height := first + offset + rand.Int63n(span)
// 주어진 에포크의 블록 높이 추정
diff := int64(epoch) - int64(now)
height = height + diff*interval
return  height, nil
}
```

설정 및 사용 예제

키매니저 워커 설정

```
# oasis-node 설정 파일
mode: keymanager

worker:
```

```yaml
keymanager:
enabled: true
runtime_id: "c0000000000000000000000000000000000000000000000000000000000000"
# 프라이빗 피어 공개키 (base64 인코딩)
private_peer_pub_keys:
- "base64-encoded-public-key-1"
- "base64-encoded-public-key-2"


runtime:
# 키매니저 런타임 번들
bundle_path: "/opt/oasis/keymanager.orc"
# SGX 설정
sgx:
loader: "/opt/oasis/oasis-core-runtime-loader"


# 합의 설정
consensus:
backend: tendermint
tendermint:
core:
listen_address: "tcp://0.0.0.0:26656"


# P2P 네트워크 설정
```

```yaml
p2p:
enabled: true
port: 9200
```

키매니저 정책 설정

```
{
"serial": 1,
"id": "c0000000000000000000000000000000000000000000000000000000000000",
"enclaves": [
{
"id": {
"mr_enclave": "hex-encoded-mrenclave",
"mr_signer": "hex-encoded-mrsigner"
},
"may_query": {
"8000000000000000000000000000000000000000000000000000000000000000": {}
},
"may_replicate": [
"hex-encoded-node-id-1",
"hex-encoded-node-id-2"
]
}
```

```
],
"master_secret_rotation_interval": 10,
"max_ephemeral_secret_age": 5
}
```

프로그래밍 방식 사용

```go
package main

import (
"context"
"fmt"
"github.com/oasisprotocol/oasis-core/go/worker/keymanager"
"github.com/oasisprotocol/oasis-core/go/worker/common"
"github.com/oasisprotocol/oasis-core/go/worker/registration"
"github.com/oasisprotocol/oasis-core/go/keymanager/api"
)

func main() {
// 공통 워커 생성
commonWorker, err := common.New(/* 설정 */)
if err != nil {
panic(fmt.Sprintf("공통 워커 생성 실패: %v", err))
```

```go
}
// 등록 워커 생성
registrationWorker, err := registration.New(/* 설정 */)
if err != nil {
panic(fmt.Sprintf("등록 워커 생성 실패: %v", err))
}
// 키매니저 백엔드 생성
keymanagerBackend, err := api.NewBackend(/* 설정 */)
if err != nil {
panic(fmt.Sprintf("키매니저 백엔드 생성 실패: %v", err))
}
// 키매니저 워커 생성
kmWorker, err := keymanager.New(
commonWorker,
registrationWorker,
keymanagerBackend,
/* 런타임 ID */,
)
if err != nil {
panic(fmt.Sprintf("키매니저 워커 생성 실패: %v", err))
}
// 워커 시작
if err := kmWorker.Start(); err != nil {
```

```
panic(fmt.Sprintf("키매니저 워커 시작 실패: %v", err))
}
// 초기화 대기
<-kmWorker.Initialized()
fmt.Println("키매니저 워커 초기화 완료")
// 상태 모니터링
go func() {
for {
status := kmWorker.GetStatus()
fmt.Printf("워커 상태: %+v\n", status)
time.Sleep(10 * time.Second)
}
}()
// 종료 대기
<-kmWorker.Quit()
fmt.Println("키매니저 워커 종료")
}
```

문제 해결

일반적인 문제

1. **엔클레이브 초기화 실패**

```
# SGX 드라이버 확인
ls -la /dev/sgx* /dev/isgx
# 엔클레이브 서명 확인
oasis-node debug sgx verify-signature runtime.sig runtime.sgxs
```

2. **키 생성 실패**

```
# 키매니저 상태 확인
oasis-node control keymanager-status --runtime-id <ID>
# 위원회 구성 확인
oasis-node control scheduler-status
```

3. **접근 제어 오류**

```
# 정책 확인
oasis-node control keymanager-policy --runtime-id <ID>
# 노드 등록 상태 확인
oasis-node control registry-node --node-id <ID>
```

디버깅 도구

```
# 상세 로그 활성화
oasis-node  --log.level  debug  --log.format  json
```

```
# 키매니저 로그 필터링
oasis-node --log.level debug 2>&1 | grep "worker/keymanager"

# 메트릭 모니터링
curl -s http://localhost:9001/metrics | grep -E "(keymanager|secret|churp)"

# 키매니저 상태 덤프
oasis-node control keymanager-status --runtime-id <ID> --verbose
```

키매니저 워커는 Oasis Core의 보안 핵심으로, 분산 키 관리와 TEE 보안을 통해 네트워크의 기밀성과 무결성을 보장합니다.